

AI 语音输入助手

项目简介

一款基于 Windows 桌面的智能语音输入工具，整合讯飞语音识别引擎与通义千问大语言模型，实现语音到文字的智能转换与优化。项目

采用 Python 语言开发，涉及音频处理、WebSocket 实时通信、RESTfulAPI 调用、系统级热键监听、GUI 开发等多个技术领域。

技术栈：Python、PyAudio、WebSocket、RESTfulAPI、多线程编程、系统托盘开发、剪贴板操作

项目地址：<https://github.com/1635338825-prog/ai-voice-input>

技术亮点

一、实时音频流处理

采用 PyAudio 实现 16kHz 采样率、16 位深度的实时音频采集，通过多线程机制实现录音与 UI 响应的并行处理，解决了音频采集阻塞主线程的问题。设计了音量实时监控机制，为用户提供录音状态反馈。

二、WebSocket 流式通信

基于讯飞语音识别 API 的 WebSocket 协议，实现了音频数据的分帧传输与实时识别结果接收。通过自定义帧状态管理（首帧、中间帧、尾帧），确保音频数据的完整传输。采用 HMAC-SHA256 签名算法实现 API 鉴权，保障通信安全。

三、大语言模型集成

对接通义千问 API，设计并优化 Prompt 工程，实现语音识别结果的智能后处理。针对语音输入场景特点，设计了包含语序修正、口语化处理、标点添加等功能的系统提示词，有效提升了输出文本的质量。

四、系统级热键监听

使用 keyboard 库实现全局热键检测，支持按住触发、松开结束的交互模式。解决了 Windows 平台下全局热键监听的权限问题，确保在各种应用场景下均能正常响应。

五、跨应用文本输入

采用剪贴板+模拟按键的技术方案，实现了跨应用程序的文本输入功能。设计了流式输入机制，逐字符输出文字，模拟真实打字效果，提升了用户体验。

六、轻量级 GUI 开发

基于 pystray 和 tkinter 实现了轻量级系统托盘应用，通过图标颜色变化和状态指示器提供实时状态反馈。采用无控制台窗口的启动方式，实现了后台静默运行。

架构设计

一、模块化设计

项目采用分层架构设计，将功能划分为核心层和表现层：

核心层：包含音频录制、语音识别、文本优化、键盘输入四个独立模块，各模块职责单一，接口清晰

表现层：包含系统托盘、状态窗口等 UI 组件，负责用户交互

二、配置管理

采用 pydantic-settings 实现类型安全的配置管理，支持从环境变量和.env 文件读取配置，实现了配置与代码的分离，便于部署和维护。

三、异步处理机制

针对语音识别和 AI 优化等耗时操作，采用多线程异步处理机制，避免阻塞 UI 线程，保证程序的响应性。使用队列进行线程间通信，确保数据传递的安全性。

四、解决的技术难点

难点一：音频设备兼容性

问题：不同电脑的麦克风设备索引不同，默认设备可能不是用户期望的麦克风。

解决方案：实现了设备枚举和选择机制，通过测试确定正确的设备索引，并在代码中提供配置接口，用户可根据实际情况调整。

难点二：WebSocket 连接稳定性

问题：网络波动可能导致 WebSocket 连接中断，影响识别体验。

解决方案：设计了完善的错误处理机制，在连接失败时能够优雅降级，返回已识别的部分结果。同时设置了合理的超时时间，避免用户长时间等待。

难点三：全局热键冲突

问题：某些应用程序可能占用相同的热键组合，导致热键失效。

解决方案：在文档中说明了管理员权限的要求，并提供了热键自定义配置，用户可根据实际情况修改快捷键组合。

难点四：跨应用输入兼容性

问题：不同应用程序对剪贴板操作和模拟按键的响应可能不同。

解决方案：采用剪贴板粘贴方式替代直接模拟按键，兼容性更好。同时设计了适当的延时机制，确保剪贴板内容更新完成后再执行粘贴操作。

性能优化

一、传输优化

增大音频帧大小，减少 WebSocket 消息发送次数

缩短帧发送间隔，加快音频数据传输速度

减少 WebSocket 连接等待时间

二、AI 调用优化

降低模型温度参数，提升输出稳定性

限制最大输出 token 数，减少响应时间

选择 qwen-turbo 模型，平衡速度与效果

四、输入优化

缩短字符间延迟，加快流式输入速度

优化剪贴板操作时序，减少不必要的等待

项目成果

实现了完整的语音输入工作流程，从录音到文字输入全流程自动化

代码结构清晰，模块划分合理，具有良好的可维护性和可扩展性

完善的错误处理机制，保证程序稳定性

详细的项目文档和配置说明，便于他人理解和使用

开源发布于 GitHub，代码规范，包含完整的 README 和配置模板

个人职责

独立完成项目的需求分析、技术选型、架构设计、编码实现、测试优化全过程

调研并对比多个语音识别方案，最终选择讯飞语音识别引擎

设计并实现 WebSocket 流式通信模块，处理实时音频传输

设计 Prompt 工程，优化 AI 文本处理效果

解决 Windows 平台下的热键监听、跨应用输入等技术问题

编写项目文档，完善开源项目规范